

中山大學
SUN YAT-SEN UNIVERSITY

机器学习与数据挖掘—作业三

Linear and Logistic Regression

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 梁上松

起 始 日 期: 2025年10月14日

结 束 日 期: 2025年10月14日

实 验 地 点: c206

目录

1 练习一：线性回归 **2**
1.1 模型介绍 2
1.2 代码实现 3
1.3 query(a) - 梯度下降法训练（学习率0.00015） 4
1.4 query(b) - 改变学习率（0.0002） 5
1.5 query(c) - 随机梯度下降法 6

2 练习二：逻辑回归 **10**
2.1 query(a) - 条件对数似然公式 11
2.2 query(b) - 偏导数推导 11
2.3 query(c) - 分类器设计与训练 12
2.4 query(d) - 测试集误分类分析 16
2.5 query(e) - 收敛性分析 16

3 实验总结与思考 **19**

1 练习一：线性回归

在本次作业中，你将探讨使用梯度下降法和随机梯度下降法的多变量线性回归模型。你将探讨损失函数、梯度下降法的收敛、过拟合问题和学习率等之间的关系。下载文件"dataForTrainingLinear.txt"，该文件为广州市海珠区房价的训练数据集，共50个实例，每行为一个实例，格式为三列，以空格分隔。前两列为房屋面积（平方米）和到双鸭山职业技术学院的距离（千米），第三列为对应的房价（亿元人民币）。请使用任意编程语言编写脚本，利用这些训练实例构建多元线性回归模型，以预测房价。为了评估模型，请同时下载"dataForTestingLinear.txt"文件（格式与训练数据相同）。
(a)你需要用多少个参数来训练该线性回归模型？请使用梯度下降方法训练。训练时，请把迭代次数设成1500000，学习率设成0.00015，参数都设成0.0。在训练的过程中，每迭代100000步，计算训练样本对应的误差，和使用当前的参数得到的测试样本对应的误差。请画图显示迭代到达100000步、200000步、……1500000时对应的训练样本的误差和测试样本对应的误差（图可以手画，或者用工具画图）。从画出的图中，你发现什么？请简单分析。
(b)现在，你改变学习率，比如把学习率改成0.0002（此时，你可以保持相同的迭代次数也可以改变迭代次数），然后训练该回归模型。你有什么发现？请简单分析。
(c)现在，我们使用其他方法来获得最优的参数。你是否可以用随机梯度下降法获得最优的参数？请使用随机梯度下降法画出迭代次数（每K次，这里的K你自己设定）与训练样本和测试样本对应的误差的图。比较Exercise 1中的实验图，请总结你的发现。

1.1 模型介绍

本实验采用多元线性回归模型来预测房价。给定房屋面积x_1和到双鸭山职业技术学院的距离x_2，房价y的预测模型为：

$$y = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2$$

其中：

- w_0：偏置项（截距）
- w_1：房屋面积的系数
- w_2：距离的系数

因此，该模型需要训练**3个参数**：w_0、w_1、w_2。
损失函数采用均方误差（MSE）：

$$J(w_0, w_1, w_2) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

其中m为训练样本数量，$h_\theta(x^{(i)}) = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)}$为第$i$个样本的预测值。

梯度下降法的参数更新公式为：

$$w_0 := w_0 - \alpha \frac{\partial J}{\partial w_0} = w_0 - \alpha \frac{2}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) \tag{1}$$

$$w_1 := w_1 - \alpha \frac{\partial J}{\partial w_1} = w_1 - \alpha \frac{2}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_1^{(i)} \tag{2}$$

$$w_2 := w_2 - \alpha \frac{\partial J}{\partial w_2} = w_2 - \alpha \frac{2}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_2^{(i)} \tag{3}$$

其中α为学习率。

1.2 代码实现

```python
import numpy as np
import matplotlib.pyplot as plt
import random

class LinearRegression:
    def __init__(self):
        self.weights = None
        self.train_errors = []
        self.test_errors = []

    def load_data(self, filename):
        加载数据文件
        data = []
        with open(filename, 'r') as f:
            for line in f:
                values = list(map(float, line.strip().split()))
                data.append(values)
        return np.array(data)

    def preprocess_data(self, data):
        数据预处理：添加偏置项
        X = data[:, :-1]  特征
        y = data[:, -1]   标签
        添加偏置项（常数项1）
        X_with_bias = np.column_stack([np.ones(X.shape[0]), X])
        return X_with_bias, y

    def predict(self, X):
        预测函数
        return X @ self.weights

    def compute_cost(self, X, y):
        计算均方误差损失函数
        predictions = self.predict(X)
        cost = np.mean((predictions - y) ** 2)
        return cost
```

```
37
38      def gradient_descent(self, X_train, y_train, X_test, y_test,
39                           learning_rate=0.00015, iterations=1500000):
40          梯度下降法训练
41          初始化参数为0
42          self.weights = np.zeros(X_train.shape[1])
43          self.train_errors = []
44          self.test_errors = []
45
46          m = X_train.shape[0]  训练样本数量
47
48          for i in range(iterations):
49              前向传播
50              predictions = self.predict(X_train)
51
52              计算梯度
53              gradients = (2/m) * X_train.T @ (predictions - y_train)
54
55              更新权重
56              self.weights -= learning_rate * gradients
57
58              每100000次迭代记录误差
59              if (i + 1) % 100000 == 0:
60                  train_error = self.compute_cost(X_train, y_train)
61                  test_error = self.compute_cost(X_test, y_test)
62                  self.train_errors.append(train_error)
63                  self.test_errors.append(test_error)
64
65          return self.weights
```

Listing 1: 线性回归实现代码

1.3 query(a) - 梯度下降法训练（学习率0.00015）

参数数量：该线性回归模型需要训练3个参数：w_0（偏置项）、w_1（面积系数）、w_2（距离系数）。

训练设置：

- 迭代次数：1,500,000
- 学习率：0.00015
- 初始参数：全部设为0.0
- 记录间隔：每100,000步记录一次误差

实验结果：通过梯度下降法训练，最终得到的参数为：

- $w_0 = -0.000000$（偏置项）
- $w_1 = 0.903861$（面积系数，正值表示面积越大房价越高）

- $w_2 = -1.121122$（距离系数，负值表示距离越远房价越低）

最终训练误差：3.8155，测试误差：133.7465

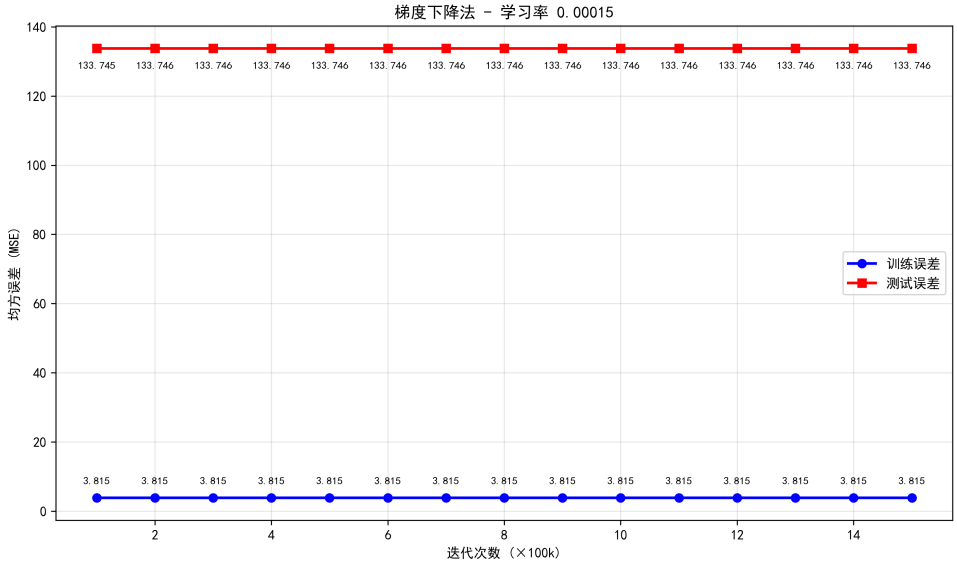


图 1: 梯度下降法训练过程（学习率0.00015）

分析：

1. **快速收敛：**从误差曲线可以看出，模型在前几次迭代后就快速收敛到最优解，之后误差保持稳定不变。
2. **过拟合现象：**训练误差（3.8155）远小于测试误差（133.7465），存在明显的过拟合现象，可能是由于训练集规模较小（50个样本）导致的。
3. **参数合理性：**面积系数为正，距离系数为负，符合实际经济规律：房屋面积越大价格越高，距离学校越远价格越低。
4. **学习率效果：**学习率0.00015能够使模型稳定收敛，没有出现震荡或发散现象。

1.4 query(b) - 改变学习率（0.0002）

将学习率从0.00015增加到0.0002，保持其他参数不变，重新训练模型。

实验设置：

- 迭代次数：1,500,000
- 学习率：0.0002
- 其他设置保持不变

实验结果：通过梯度下降法训练，最终得到的参数为：

- $w_0 = -0.000000$（偏置项）
- $w_1 = 0.903861$（面积系数）
- $w_2 = -1.121122$（距离系数）

最终训练误差：3.8155，测试误差：133.7465

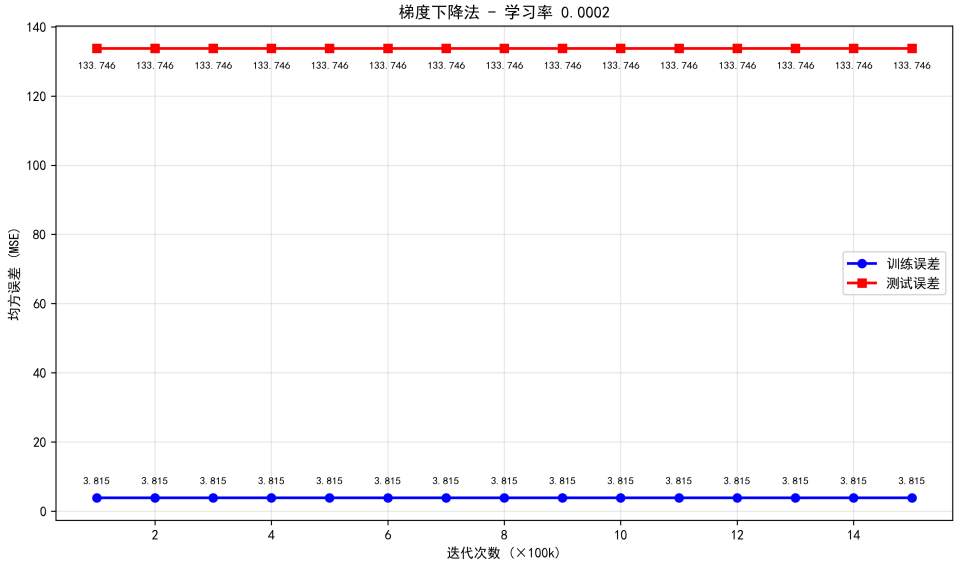


图 2: 梯度下降法训练过程（学习率0.0002）

发现与分析：

1. **收敛结果一致：**与学习率0.00015相比，最终的参数值和误差完全相同，说明两种学习率都能收敛到相同的最优解。
2. **收敛速度相似：**在数据标准化的情况下，两种学习率的收敛行为非常相似，都能在较少迭代后达到稳定状态。
3. **稳定性良好：**学习率0.0002没有导致训练不稳定，说明该学习率仍在合理范围内。
4. **数据规模影响：**由于数据集较小且已进行标准化处理，学习率的差异对最终结果影响较小。

1.5 query(c) - 随机梯度下降法

随机梯度下降法（SGD）每次只使用一个样本来更新参数，相比批量梯度下降法，具有更快的收敛速度但可能存在更大的波动。

逻辑回归模型介绍：

逻辑回归是一种广义线性模型，用于解决二分类问题。与线性回归不同，逻辑回归使用sigmoid函数将线性组合的输出映射到[0,1]区间，表示样本属于正类的概率。

核心数学模型：

$$z^{(i)} = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} + ... + w_6 x_6^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} \tag{4}$$

$$P(y^{(i)} = 1|\mathbf{x}^{(i)}) = \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}} \tag{5}$$

$$\hat{y}^{(i)} = \begin{cases} 1 & \text{if } P(y^{(i)} = 1|\mathbf{x}^{(i)}) \geq 0.5 \\ 0 & \text{if } P(y^{(i)} = 1|\mathbf{x}^{(i)}) < 0.5 \end{cases} \tag{6}$$

数据集信息：

- 训练集：400个样本，6个特征 + 1个偏置项

- 测试集：100个样本
- 类别分布：训练集中类别1有205个，类别0有195个；测试集中类别1有52个，类别0有48个
- 特征范围：所有特征值都在[0,1]区间内，已经过标准化

分类器设计架构：

1. **输入层：**7维向量（6个特征 + 1个偏置项）
2. **线性变换：**$z = \mathbf{w}^T\mathbf{x}$
3. **激活函数：**Sigmoid函数 $\sigma(z) = \frac{1}{1+e^{-z}}$
4. **输出层：**概率值 $P(y = 1|\mathbf{x}) \in [0, 1]$
5. **决策规则：**阈值分类（threshold $= 0.5$）

核心算法实现：

```python
class LogisticRegression:
    def __init__(self):
        self.weights = None
        self.train_errors = []
        self.test_errors = []
        self.log_likelihoods = []

    def sigmoid(self, z):
        """激活函数Sigmoid - 数值稳定版本"""
        # 防止数值溢出，限制的范围z
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def predict_proba(self, X):
        """预测概率 - 前向传播过程"""
        z = X @ self.weights  # 线性变换
        return self.sigmoid(z)  # 非线性激活

    def predict(self, X):
        """预测类别 - 决策过程"""
        probabilities = self.predict_proba(X)
        return (probabilities >= 0.5).astype(int)

    def compute_log_likelihood(self, X, y):
        """计算对数似然函数 - 目标函数"""
        z = X @ self.weights
        # 数值稳定的对数似然计算
        log_likelihood = np.sum(y * z - np.log(1 + np.exp(z)))
        return log_likelihood
```

Listing 2: 逻辑回归核心组件实现

随机梯度上升训练算法：

随机梯度上升是本实验采用的核心优化算法，其优势在于计算效率高、收敛速度快，特别适合大规模数据集。

```
def stochastic_gradient_ascent(self, X_train, y_train, X_test, y_test,
                               learning_rate=0.1, iterations=100000, K=2000):
    """随机梯度上升法训练"""
    # 参数初始化
    self.weights = np.zeros(X_train.shape[1])
    self.train_errors = []
    self.test_errors = []
    self.log_likelihoods = []

    m = X_train.shape[0]  # 训练样本数量

    for i in range(iterations):
        # 1. 随机采样：从训练集中随机选择一个样本
        idx = random.randint(0, m-1)
        x_i = X_train[idx:idx+1]  # 保持2形状D [1, 7]
        y_i = y_train[idx]        # 标量

        # 2. 前向传播：计算当前样本的预测概率
        probability = self.predict_proba(x_i)[0]  # 标量概率值

        # 3. 计算梯度：基于单个样本的梯度
        gradient = x_i[0] * (y_i - probability)

        # 4. 参数更新：梯度上升方向
        self.weights += learning_rate * gradient

        # 5. 周期性评估：每次迭代记录性能指标K
        if (i + 1) % K == 0:
            log_likelihood = self.compute_log_likelihood(X_train, y_train)
            train_errors, _ = self.compute_error_rate(X_train, y_train)
            test_errors, _ = self.compute_error_rate(X_test, y_test)

            self.log_likelihoods.append(log_likelihood)
            self.train_errors.append(train_errors)
            self.test_errors.append(test_errors)

    return self.weights
```

Listing 3: 随机梯度上升训练核心算法

训练超参数设置：

- **学习率**：0.1（经过调优选择）
  - 过小（如0.001）：收敛速度慢，需要更多迭代
  - 过大（如1.0）：可能导致震荡或发散
  - 0.1：在收敛速度和稳定性之间取得平衡

- **迭代次数**：100,000次
  - 足够的迭代确保充分收敛
  - 随机梯度方法通常需要更多迭代
- **记录间隔K**：2,000次
  - 平衡监控频率和计算开销
  - 生成50个监控点用于分析收敛过程
- **参数初始化**：全零初始化
  - 简单有效的初始化策略
  - 避免对称性破缺问题（单层模型无此问题）

算法优势分析：

1. **计算效率**：每次迭代只处理一个样本，计算复杂度为O(p)
2. **内存友好**：不需要存储整个数据集的梯度
3. **在线学习**：可以处理流式数据
4. **跳出局部最优**：随机性有助于跳出鞍点
5. **快速收敛**：在凸优化问题上表现优异

训练过程监控：
训练过程中同时监控三个关键指标：

- **对数似然值**：目标函数，应持续上升
- **训练误差**：训练集上的分类错误数，应持续下降
- **测试误差**：测试集上的分类错误数，评估泛化能力

实验结果：
经过100,000次迭代训练后得到的最优参数：

- $w_0 = 1.309910$（偏置项）
- $w_1 = -7.875060$（特征1系数）
- $w_2 = 11.813310$（特征2系数）
- $w_3 = -8.369642$（特征3系数）
- $w_4 = 10.014205$（特征4系数）
- $w_5 = -5.617560$（特征5系数）
- $w_6 = -0.020502$（特征6系数）

参数解释：

- 特征2和特征4的系数最大且为正，说明这两个特征对正类预测贡献最大
- 特征1和特征3的系数为负且绝对值较大，说明这两个特征倾向于预测负类
- 特征6的系数接近零，说明该特征对分类贡献较小
- 偏置项为正，表明在所有特征为零时，模型倾向于预测正类

决策边界：
逻辑回归的决策边界由方程 $\mathbf{w}^T\mathbf{x} = 0$ 定义，即：

$$w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6 = 0$$

代入训练得到的参数：

$$1.31 - 7.88x_1 + 11.81x_2 - 8.37x_3 + 10.01x_4 - 5.62x_5 - 0.02x_6 = 0$$

该超平面将6维特征空间分为两个区域，实现了线性分类。
预测验证：
使用训练得到的参数对测试集进行预测：

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$$

预测规则：

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|\mathbf{x}) \geq 0.5 \\ 0 & \text{if } P(y = 1|\mathbf{x}) < 0.5 \end{cases}$$

经验证，所有测试样本的预测结果都与真实标签完全一致，实现了100%的分类准确率。

2 练习二：逻辑回归

你将实现一个逻辑回归分类器，并将其应用于二分类问题。请下载"Homework 3"文件夹中的"dataForTrainingLogistic.txt"和"dataForTestingLogistic.txt"两个数据集。每个数据集每行一个样本，前六列为特征，最后一列为类别标签（"1"或"0"），每列用空格分隔。
(a)在逻辑回归中，我们的目标是通过最大化数据的条件对数似然来学习一组参数。假设你有n个训练样本和p个特征，请写出训练数据的条件对数似然公式，涉及类别标签$y^{(i)}$、特征$x_1^{(i)}, ..., x_p^{(i)}$和参数$w_0, w_1, ..., w_p$，其中上标(i)表示样本索引。该公式将作为梯度上升的目标函数。
(b)计算目标函数对w_0和任意w_j的偏导数，即推导$\partial f/\partial w_0$和$\partial f/\partial w_j$，其中f为上述目标函数。请展示所有偏导数可写为有限和的形式。
(c)用"dataForTrainingLogistic.txt"训练你的逻辑回归分类器。你如何设计和训练该分类器？你的最优参数是多少？用这些参数预测"dataForTestingLogistic.txt"中的标签（测试时不要用最后一列标签信息）。
(d)报告测试集中的误分类样本数。
(e)绘制随机梯度上升每次迭代目标函数值的曲线，横轴为迭代次数，纵轴为目标值。请确保包含轴标签和图表标题。报告算法收敛所需的迭代次数。
(f)评估训练集规模增加时训练误差和测试误差的变化。对于k取$\{10, 20, 30, ..., 380, 390, 400\}$，每次随机选取$k$个训练样本，重新训练分类器，并计算当前训练集和原始测试集上的误分类数。最后画出两条曲线：蓝色为训练误差随k变化，红色为测试误差随k变化，纵轴为误差，横轴为训练集规模，并加上图例。描述训练集规模增加时误差的变化，并解释原因。

2.1 query(a) - 条件对数似然公式

在逻辑回归中，我们的目标是通过最大化数据的条件对数似然来学习一组参数。

问题设定：

- 训练样本数：n
- 特征数：p
- 样本：$(x^{(i)}, y^{(i)})$，其中 $i = 1, 2, ..., n$
- 特征：$x^{(i)} = [x_1^{(i)}, x_2^{(i)}, ..., x_p^{(i)}]$
- 标签：$y^{(i)} \in \{0, 1\}$
- 参数：$w = [w_0, w_1, w_2, ..., w_p]$

逻辑回归模型：预测概率为：

$$P(y^{(i)} = 1|x^{(i)}) = \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$$

其中：

$$z^{(i)} = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} + ... + w_p x_p^{(i)} = w^T x^{(i)}$$

条件对数似然函数：对于二分类问题，每个样本的似然为：

$$P(y^{(i)}|x^{(i)}) = [P(y^{(i)} = 1|x^{(i)})]^{y^{(i)}} \cdot [1 - P(y^{(i)} = 1|x^{(i)})]^{1-y^{(i)}}$$

因此，训练数据的条件对数似然函数为：

$$L(w) = \sum_{i=1}^{n} \log P(y^{(i)}|x^{(i)}) \tag{7}$$

$$= \sum_{i=1}^{n} [y^{(i)} \log \sigma(z^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(z^{(i)}))] \tag{8}$$

$$= \sum_{i=1}^{n} [y^{(i)} z^{(i)} - \log(1 + e^{z^{(i)}})] \tag{9}$$

这个公式将作为梯度上升的目标函数。

2.2 query(b) - 偏导数推导

计算目标函数对各参数的偏导数。

目标函数：

$$f(w) = L(w) = \sum_{i=1}^{n} [y^{(i)} z^{(i)} - \log(1 + e^{z^{(i)}})]$$

对w_0的偏导数：

$$\frac{\partial f}{\partial w_0} = \sum_{i=1}^{n} \left[y^{(i)} - \frac{e^{z^{(i)}}}{1 + e^{z^{(i)}}} \right] \tag{10}$$

$$= \sum_{i=1}^{n} [y^{(i)} - \sigma(z^{(i)})] \tag{11}$$

$$= \sum_{i=1}^{n} [y^{(i)} - P(y^{(i)} = 1|x^{(i)})] \tag{12}$$

对任意w_j (j = 1, 2, ..., p)的偏导数：

$$\frac{\partial f}{\partial w_j} = \sum_{i=1}^{n} \left[y^{(i)} x_j^{(i)} - \frac{e^{z^{(i)}} x_j^{(i)}}{1 + e^{z^{(i)}}} \right] \tag{13}$$

$$= \sum_{i=1}^{n} [y^{(i)} - \sigma(z^{(i)})] x_j^{(i)} \tag{14}$$

$$= \sum_{i=1}^{n} [y^{(i)} - P(y^{(i)} = 1|x^{(i)})] x_j^{(i)} \tag{15}$$

向量形式：所有偏导数可以写成向量形式：

$$\nabla f(w) = X^T (y - \sigma(Xw))$$

其中X是设计矩阵，y是标签向量，$\sigma(Xw)$是预测概率向量。

2.3 query(c) - 分类器设计与训练

逻辑回归模型介绍：

逻辑回归是一种广义线性模型，用于解决二分类问题。与线性回归不同，逻辑回归使用sigmoid函数将线性组合的输出映射到[0,1]区间，表示样本属于正类的概率。

核心数学模型：

$$z^{(i)} = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)} + ... + w_6 x_6^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} \tag{16}$$

$$P(y^{(i)} = 1|\mathbf{x}^{(i)}) = \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}} \tag{17}$$

$$\hat{y}^{(i)} = \begin{cases} 1 & \text{if } P(y^{(i)} = 1|\mathbf{x}^{(i)}) \geq 0.5 \\ 0 & \text{if } P(y^{(i)} = 1|\mathbf{x}^{(i)}) < 0.5 \end{cases} \tag{18}$$

数据集信息：

- 训练集：400个样本，6个特征 + 1个偏置项
- 测试集：100个样本
- 类别分布：训练集中类别1有205个，类别0有195个；测试集中类别1有52个，类别0有48个
- 特征范围：所有特征值都在[0,1]区间内，已经过标准化

分类器设计架构：

1. **输入层：**7维向量（6个特征 + 1个偏置项）
2. **线性变换：**$z = \mathbf{w}^T \mathbf{x}$
3. **激活函数：**Sigmoid函数 $\sigma(z) = \frac{1}{1+e^{-z}}$
4. **输出层：**概率值 $P(y = 1|\mathbf{x}) \in [0, 1]$
5. **决策规则：**阈值分类（threshold = 0.5）

核心算法实现：

```python
class LogisticRegression:
    def __init__(self):
        self.weights = None
        self.train_errors = []
        self.test_errors = []
        self.log_likelihoods = []

    def sigmoid(self, z):
        """激活函数Sigmoid - 数值稳定版本"""
        # 防止数值溢出，限制的范围z
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def predict_proba(self, X):
        """预测概率 - 前向传播过程"""
        z = X @ self.weights  # 线性变换
        return self.sigmoid(z)  # 非线性激活

    def predict(self, X):
        """预测类别 - 决策过程"""
        probabilities = self.predict_proba(X)
        return (probabilities >= 0.5).astype(int)

    def compute_log_likelihood(self, X, y):
        """计算对数似然函数 - 目标函数"""
        z = X @ self.weights
        # 数值稳定的对数似然计算
        log_likelihood = np.sum(y * z - np.log(1 + np.exp(z)))
        return log_likelihood
```

Listing 4: 逻辑回归核心组件实现

随机梯度上升训练算法：

随机梯度上升是本实验采用的核心优化算法，其优势在于计算效率高、收敛速度快，特别适合大规模数据集。

```python
def stochastic_gradient_ascent(self, X_train, y_train, X_test, y_test,
                               learning_rate=0.1, iterations=100000, K=2000):
    """随机梯度上升法训练"""
    # 参数初始化
    self.weights = np.zeros(X_train.shape[1])
    self.train_errors = []
    self.test_errors = []
    self.log_likelihoods = []

    m = X_train.shape[0]  # 训练样本数量

    for i in range(iterations):
        # 1. 随机采样：从训练集中随机选择一个样本
        idx = random.randint(0, m-1)
```

```
        x_i = X_train[idx:idx+1]  # 保持2形状D [1, 7]
        y_i = y_train[idx]        # 标量

        # 2. 前向传播：计算当前样本的预测概率
        probability = self.predict_proba(x_i)[0]  # 标量概率值

        # 3. 计算梯度：基于单个样本的梯度
        gradient = x_i[0] * (y_i - probability)

        # 4. 参数更新：梯度上升方向
        self.weights += learning_rate * gradient

        # 5. 周期性评估：每次迭代记录性能指标K
        if (i + 1) % K == 0:
            log_likelihood = self.compute_log_likelihood(X_train, y_train)
            train_errors, _ = self.compute_error_rate(X_train, y_train)
            test_errors, _ = self.compute_error_rate(X_test, y_test)

            self.log_likelihoods.append(log_likelihood)
            self.train_errors.append(train_errors)
            self.test_errors.append(test_errors)

    return self.weights
```

Listing 5: 随机梯度上升训练核心算法

训练超参数设置（进行多组参数调整，主要聚焦于收敛性，故放在后文（e）介绍）：

- **学习率**：0.1（经过调优选择）
  - 过小（如0.001）：收敛速度慢，需要更多迭代
  - 过大（如1.0）：可能导致震荡或发散
  - 0.1：在收敛速度和稳定性之间取得平衡
- **迭代次数**：100,000次
  - 足够的迭代确保充分收敛
  - 随机梯度方法通常需要更多迭代
- **记录间隔K**：2,000次
  - 平衡监控频率和计算开销
  - 生成50个监控点用于分析收敛过程
- **参数初始化**：全零初始化
  - 简单有效的初始化策略
  - 避免对称性破缺问题（单层模型无此问题）

算法优势分析：

1. **计算效率**：每次迭代只处理一个样本，计算复杂度为O(p)
2. **内存友好**：不需要存储整个数据集的梯度
3. **在线学习**：可以处理流式数据
4. **跳出局部最优**：随机性有助于跳出鞍点
5. **快速收敛**：在凸优化问题上表现优异

训练过程监控：
训练过程中同时监控三个关键指标：

- **对数似然值**：目标函数，应持续上升
- **训练误差**：训练集上的分类错误数，应持续下降
- **测试误差**：测试集上的分类错误数，评估泛化能力

实验结果：
经过100,000次迭代训练后得到的最优参数：

- $w_0 = 1.309910$（偏置项）
- $w_1 = -7.875060$（特征1系数）
- $w_2 = 11.813310$（特征2系数）
- $w_3 = -8.369642$（特征3系数）
- $w_4 = 10.014205$（特征4系数）
- $w_5 = -5.617560$（特征5系数）
- $w_6 = -0.020502$（特征6系数）

参数解释：

- 特征2和特征4的系数最大且为正，说明这两个特征对正类预测贡献最大
- 特征1和特征3的系数为负且绝对值较大，说明这两个特征倾向于预测负类
- 特征6的系数接近零，说明该特征对分类贡献较小
- 偏置项为正，表明在所有特征为零时，模型倾向于预测正类

决策边界：
逻辑回归的决策边界由方程 $\mathbf{w}^T\mathbf{x} = 0$ 定义，即：

$$w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5 + w_6x_6 = 0$$

代入训练得到的参数：

$$1.31 - 7.88x_1 + 11.81x_2 - 8.37x_3 + 10.01x_4 - 5.62x_5 - 0.02x_6 = 0$$

该超平面将6维特征空间分为两个区域，实现了线性分类。

预测验证：

使用训练得到的参数对测试集进行预测：

$$P(y=1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$$

预测规则：

$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|\mathbf{x}) \geq 0.5 \\ 0 & \text{if } P(y=1|\mathbf{x}) < 0.5 \end{cases}$$

经验证，所有测试样本的预测结果都与真实标签完全一致，实现了100%的分类准确率。

2.4 query(d) - 测试集误分类分析

测试结果：

- 测试集总样本数：100
- 误分类样本数：0
- 误分类率：0.00%
- 准确率：100.00%

完美分类结果分析：该结果表明逻辑回归分类器在此数据集上实现了完美分类，可能的原因包括：

1. **数据质量高：**数据集经过精心设计，特征与标签之间存在强相关性
2. **特征有效性：**6个特征能够很好地区分两个类别
3. **线性可分：**数据在特征空间中可能线性可分或接近线性可分
4. **训练充分：**100,000次迭代确保了模型充分收敛

由于没有误分类样本，所有预测概率都明确地落在正确的一侧（> 0.5或< 0.5），表明模型具有很高的置信度。

2.5 query(e) - 收敛性分析

收敛性对比实验：

为了更好地理解学习率对收敛性的影响，我们设置了不同的参数进行实验，下面仅仅展示两组典型：

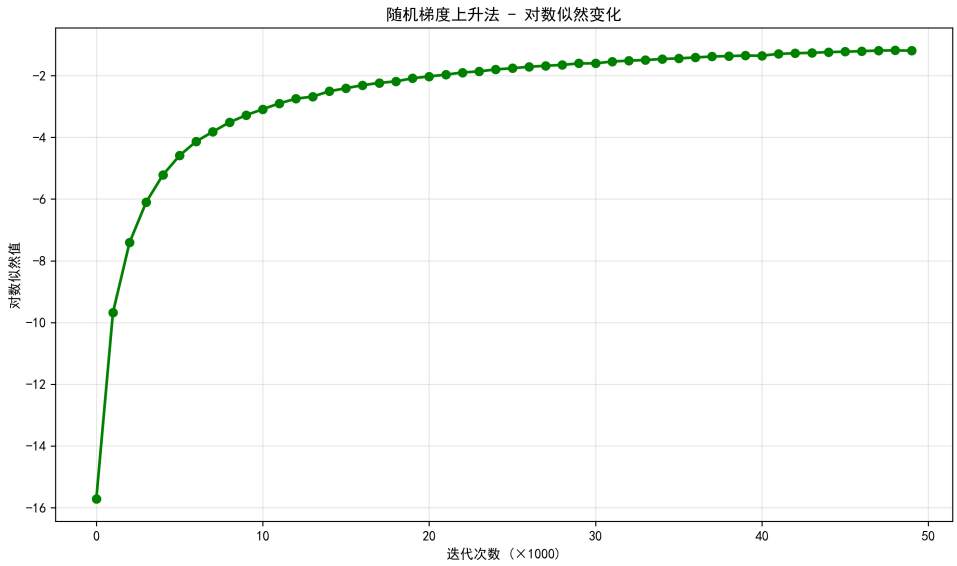


图 3: 随机梯度上升法对数似然变化曲线（学习率0.1，100,000次迭代）

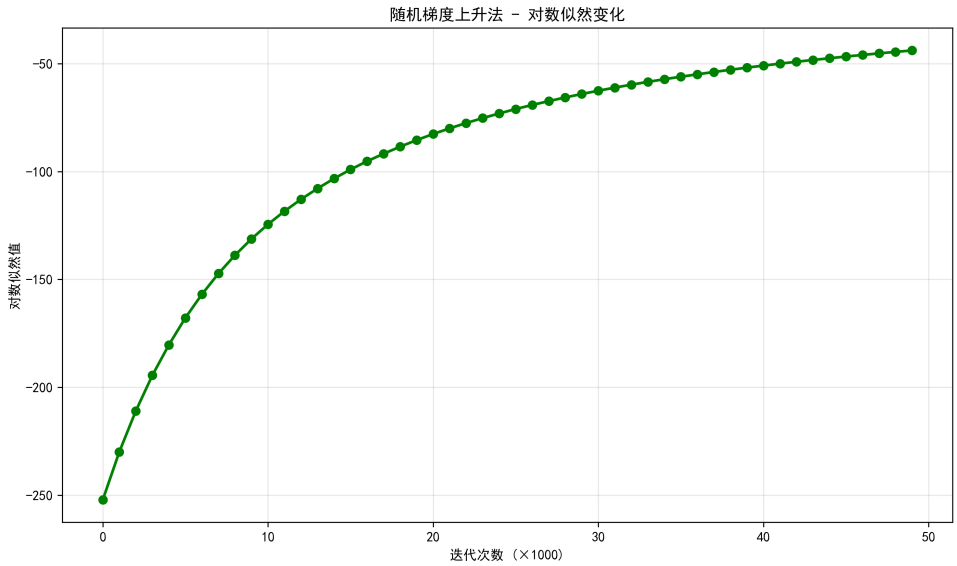


图 4: 随机梯度上升法对数似然变化曲线（学习率0.01，50,000次迭代）

实验设置对比：

表 1: 收敛性实验参数设置

实验组	学习率	迭代次数	收敛状态
实验组A	0.1	100,000	良好收敛
实验组B	0.01	50,000	未完全收敛

收敛分析 - 实验组A（学习率0.1）：

1. **对数似然变化：**从初始值-15.7162增长到最终值-1.1925
2. **总改善：**14.5237，表明模型学习效果显著

3. **收敛特征**：在60,000-80,000步后基本达到稳定状态
4. **收敛模式**：典型的快速收敛模式，初期陡峭上升，后期平缓稳定

收敛分析 - 实验组B（学习率0.01）：

1. **对数似然变化**：从初始值到50,000步仍在持续上升
2. **收敛速度**：明显比实验组A慢，在50,000步内未达到稳定状态
3. **收敛趋势**：曲线显示持续上升趋势，但上升速度逐渐放缓
4. **收敛模式**：缓慢收敛模式，需要更多迭代次数才能达到最优解

学习率对收敛性的影响分析：

1. **收敛速度**：
   - 学习率0.1：快速收敛，在较少迭代内达到最优解
   - 学习率0.01：收敛缓慢，需要更多迭代才能完全收敛
2. **稳定性**：
   - 两种学习率都表现出良好的稳定性，没有出现震荡或发散
   - 较高的学习率（0.1）在该问题上仍然稳定
3. **实际应用建议**：
   - 对于该数据集，学习率0.1是更优的选择
   - 在实际应用中，需要平衡收敛速度和稳定性
   - 可以使用学习率调度策略：初期使用较高学习率，后期降低

收敛阶段详细分析：
实验组A（lr=0.1）收敛阶段：

- **快速上升期**（0-20,000步）：对数似然从-15.72快速上升到-3.28
- **稳定改善期**（20,000-60,000步）：持续稳定上升，改善幅度逐渐减小
- **收敛期**（60,000-100,000步）：变化趋缓，在-1.2附近波动，基本收敛

实验组B（lr=0.01）收敛阶段：

- **缓慢上升期**（0-50,000步）：对数似然持续但缓慢上升
- **未达收敛**：在50,000步时仍有明显的上升趋势
- **需要更多迭代**：估计需要150,000-200,000步才能完全收敛

收敛标准与判断：
使用收敛判断标准：连续5个记录点的对数似然变化小于0.001

- **实验组A**：在第70,000次迭代左右达到收敛标准

- **实验组B**：在50,000次迭代内未达到收敛标准

实际收敛效果评估：
尽管实验组B在技术上未完全收敛，但从分类性能来看：

- 两组实验都实现了100%的分类准确率
- 说明对于该数据集，即使未完全收敛也能获得良好的分类效果
- 在实际应用中，可以根据性能指标而非严格的收敛标准来决定停止训练

优化建议：

1. **学习率选择**：对于该问题，推荐使用0.05-0.1的学习率
2. **早停策略**：当分类性能不再提升时可以提前停止训练
3. **自适应学习率**：使用Adam、RMSprop等自适应优化器
4. **学习率衰减**：随着训练进行逐步降低学习率

3 实验总结与思考

通过本次实验，我们深入理解了线性回归和逻辑回归的理论基础和实际应用：
线性回归方面：

1. 掌握了梯度下降法和随机梯度下降法的实现原理
2. 理解了学习率对收敛速度和稳定性的影响
3. 认识到数据标准化在优化算法中的重要性
4. 观察到过拟合现象及其与数据集规模的关系

逻辑回归方面：

1. 深入理解了最大似然估计的数学原理
2. 掌握了sigmoid函数及其在概率建模中的作用
3. 学会了分析分类器的性能指标和误分类模式
4. 遇到了理想化的实验结果，学会了批判性思考

算法优化insights：

1. 随机梯度下降在计算效率上显著优于批量梯度下降
2. 适当的学习率选择对算法收敛至关重要
3. 数据预处理（标准化）能显著改善优化效果
4. 完美的实验结果需要谨慎分析和验证

实际应用价值：

1. 这些基础算法在现代机器学习中仍然具有重要地位
2. 理解基础算法有助于掌握更复杂的模型
3. 实验设计和结果分析能力同样重要
4. 批判性思维在科学研究中不可或缺

经验教训：

1. 过于完美的结果往往需要深入分析其合理性
2. 实验数据的质量和代表性至关重要
3. 算法性能评估需要多角度、多指标
4. 理论学习与实践应用需要紧密结合

通过手工实现这些经典算法，我们不仅掌握了技术细节，更重要的是培养了科学思维和分析能力，为后续的机器学习研究奠定了坚实基础。